

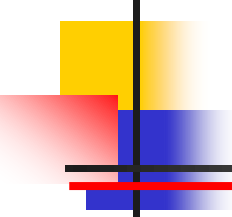
Introducción a la computación

SESION 10

Estructuras Repetitivas

For

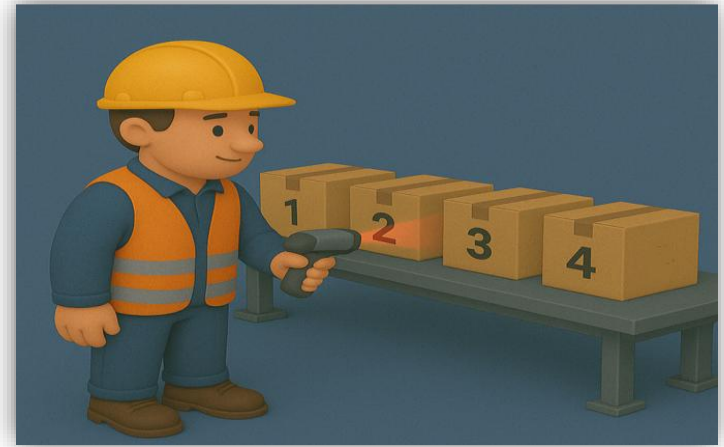
UNIVERSIDAD NACIONAL DE INGENIERIA
Facultad de Ingeniería Industrial y de Sistemas
Ing. Miguel Angel Navarro Neyra
mnavarron@uni.edu.pe



¿Qué es una estructura iterativa?

- Una **estructura iterativa**, también llamada **bucle** o **ciclo**, es una **construcción de programación** que permite ejecutar un **bloque de instrucciones varias veces**, hasta que se cumpla una **condición de salida** o se **alcance un número determinado de repeticiones**.
 - Sirve para **automatizar** tareas repetitivas, **reduciendo líneas de código** y **errores**.
-

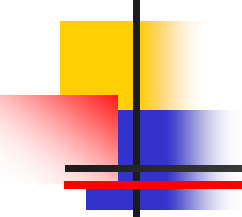
¿Qué de común tiene estas imágenes de la vida real?



Ambas imágenes ilustran claramente cómo una **acción repetitiva sobre elementos ordenados** puede representarse con una **estructura de repetición en programación**, particularmente un **bucle for** que recorre una secuencia.

Características de la Estructura Repetitiva for

Característica	for
Definición	Permite repetir un bloque de instrucciones un número determinado de veces. Utiliza una variable que se inicializa, evalúa una condición y se actualiza en cada iteración.
Uso típico	Cuando se conoce la cantidad de repeticiones.
Sintaxis básica	<pre>for (inicialización; condición; incremento) { instrucciones; }</pre>
Ejemplo	<pre>for (int i = 1; i < 6; i++) { cout << i << endl; }</pre>



Ejemplo con for

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i < 5; i++) {
        cout << "Número " << i << endl;
    }

    return 0;
}
```

i = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4

Número 0
Número 1
Número 2
Número 3
Número 4

Recorrer una lista con for

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> lista = {2, 3, 5, 7};

    for (int elemento : lista) {
        int y = elemento * elemento; // elevar al cuadrado
        cout << y << endl;
    }

    return 0;
}
```

elemento = ~~2~~ ~~3~~ ~~5~~ 7
y = ~~4~~ ~~9~~ ~~25~~ 49

4
9
25
49



Break en for

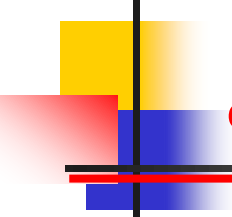
```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> numeros = {10, 20, 55, 30, 40};

    for (int n : numeros) {
        if (n > 50) {
            cout << "Número mayor a 50 encontrado: " << n << endl;
            break;
        }
    }

    return 0;
}
```

El **break** detiene el bucle inmediatamente cuando se cumple una condición. En este caso, cuando se encuentra un número **mayor a 50**, ya no se revisan más elementos.



continue en for

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> numeros = {1, 2, 3, 4, 5};

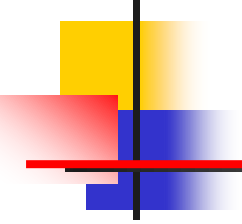
    for (int n : numeros) {
        if (n % 2 != 0) {
            continue; // salta los impares
        }
        cout << "Número par: " << n << endl;
    }

    return 0;
}
```

El **continue** salta esa iteración **si se cumple la condición** (en este caso, si el número es impar), y continúa con el siguiente valor.

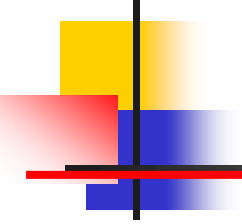


Ejercicios



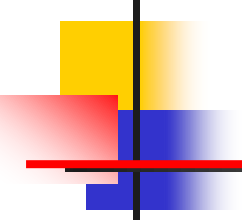
Ejemplo 01

Cree un algoritmo que permita registrar las temperaturas, en grados Celsius, de un horno industrial durante su funcionamiento continuo. El sistema debe pedir cada temperatura registrada hasta que se detecte una temperatura mayor a 90 °C, momento en el cual se interrumpe el registro debido a una posible condición de riesgo térmico. El programa debe un mensaje "Alerta registro terminado se excedió temperatura límite" y mostrar en una lista todas las temperaturas válidas ingresadas antes de la interrupción.



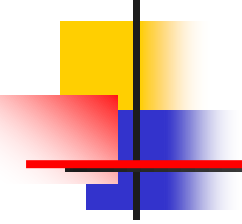
Ejemplo 02

Solicitar un número entero mayor o igual a cero (validar). La aplicación mostrará la factorial del número.



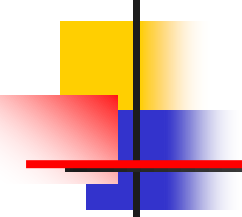
Ejemplo 03

Solicitar un número entero de 4 dígitos (validar). La aplicación deberá mostrar la suma de sus dígitos.



Ejemplo 04

Contar y mostrar la cantidad de divisores de un número n ingresado por teclado. El valor de n deberá ser mayor que cero.



Ejemplo 05

Una fábrica de varillas circulares de fierro efectúa el control de calidad de la siguiente manera:

Para aceptar o rechazar un lote de producción, extrae n varillas de dicho lote y mide el diámetro de cada una de ellas. Si más del 10% de las n varillas tiene diámetro fuera del rango $[1.5 \text{ cm}-2.0 \text{ cm}]$, el lote es rechazado.

Desarrolle una aplicación que solicite el valor de n y luego solicite los diámetros de las N varillas. La aplicación debe mostrar:

- a) El diámetro promedio
 - b) El número de varillas fuera del rango $[1.5 \text{ cms}-2.0 \text{ cms}]$
 - c) Si el lote es rechazado o aceptado.
-



Ejemplo 06

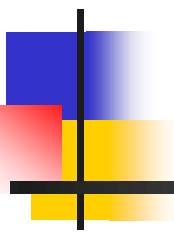
Una empresa abastece 2 tipos de uva. Desarrolle una aplicación para registrar 7 pedidos de uva. Para cada pedido se ingresarán los siguientes datos:

- Tipo de uva (1: Borgoña o 2: Italia. En cualquier otro caso volverá a solicitar el dato hasta que sea correcto).
- Peso en toneladas. (Considerar que es un número entero mayor a cero, en cualquier otro caso volverá a solicitar el dato hasta que sea correcto).

Luego de terminar de procesar los 7 pedidos se deberá mostrar un reporte con la siguiente información:

- a) El número de viajes requerido para trasladar el total de toneladas de uva Borgoña (de los 7 pedidos) con un camión de 40 toneladas de capacidad máxima. Para que la solución sea válida deberá utilizar los operadores de división entera y residuo de la división entera. Por ejemplo: si en total por los 7 pedidos se solicitó 9722 toneladas de uva, la empresa requerirá 244 viajes.
 - b) El número de pedidos de uva Italia.
-

Introducción a la computación



FIN SESION 11